Advanced Data Base Systems (22MCA102)

Procedural Programming with MySQL:

BY

Dr. Anantha Murthy
Associate Professor
Dept. of MCA
NMAM.I.T, Nitte

Unit 3

1. Introduction to Stored Programs

Introduction to Stored Programming

- MySQL provides for using standard SQL to write stored programs. Stored programs can include procedural code that controls the flow of execution.

- Stored programs in MySQL are **precompiled SQL code blocks** that are **saved in the database** and can be **executed on demand**. They allow you to move the business logic closer to the data, reducing network traffic and improving efficiency.

- These programs can include control-flow logic (like IF, LOOP, CASE), variable declarations, and even error handling — similar to a programming language, but executed within the MySQL server.

Why Stored Programming

🔁 1. **Promote Code Reuse**

- You can write a procedure or function once and call it many times.
- Reduces repetition and enforces consistency in business logic.

🚀 2. **Improve Performance**

- Stored programs are compiled and cached by the server.
- Execution occurs **within the database server**, minimizing round-trips between client and server.

🔐 3. **Enhance Security**

- Users can be **granted access to procedures/functions** without having direct access to the underlying tables.
- Ensures data abstraction and protection.

🧩 4. **Encapsulate Business Logic**

- Logic such as payroll calculation, order processing, validations, etc., can be built into the database layer.
- Easier to maintain and update logic centrally.

Types of Stored Programs in MySQL

1. Stored Procedures

- A set of SQL statements grouped together under a name. May accept input/output parameters.
- Used to perform tasks or processes.

2. Stored Functions

- Similar to procedures but must return a single value.
- Can be used inside SQL expressions like SELECT, WHERE.

3. Triggers

- A block of SQL code that automatically executes in response to specific table events like: BEFORE INSERT, AFTER UPDATE, etc.
- Useful for enforcing rules, auditing, logging.

Stored Procedures

A Stored Procedure is a named block of SQL statements that are stored in the database and can be executed repeatedly with a single CALL statement.

It may accept parameters (IN, OUT, INOUT), and can include procedural logic (loops, conditions, cursors, etc.).

Syntax:

```
DELIMITER //

CREATE PROCEDURE procedure_name ([parameters])
BEGIN
    -- SQL statements;
END;
//

DELIMITER ;
```

Creating and Calling Stored Procedures

- `DELIMITER //` : Temporarily changes the SQL statement terminator to `//` to allow multiline statements.
- `CREATE PROCEDURE` : Begins the definition of the stored procedure.
- `[parameters]` : Optional — can include:
  - `IN` : Accepts input values.
  - `OUT` : Returns values to the caller.
  - `INOUT` : Both input and output.
- `BEGIN...END` : Encapsulates the body of the procedure.
- `SQL statements` : Any valid SQL or procedural statements.

Stored Procedure Without Parameters

```
DELIMITER //

CREATE PROCEDURE ShowAllEmployees()
BEGIN
    SELECT id AS "Employee ID", name AS "Employee Name"
    FROM employees;
END;
//

DELIMITER ;
```

Calling the Stored Procedure:

```
CALL ShowAllEmployees();
```

Output

```
+-------------+-----------------+
| Employee ID | Employee Name   |
+-------------+-----------------+
|     101     | Alice Kumar     |
|     102     | Rahul Singh     |
|     103     | Sneha Menon     |
|     104     | John D'Souza    |
|     105     | Priya Rao       |
+-------------+-----------------+
```

How to Code Input and Output Parameters

MySQL allows you to define **parameters** in stored procedures to pass values **into** and **out of** the procedure.

There are **three types** of parameters:

Type	Direction	Usage
`IN`	Input to procedure	Default; passed by value
`OUT`	Output from procedure	Returns data to the caller
`INOUT`	Input and output both	Accepts and returns values

How to Code Input and Output Parameters

Syntax for Stored Procedure with Parameters

```
CREATE PROCEDURE procedure_name(
    IN param1 datatype,
    OUT param2 datatype,
    INOUT param3 datatype
)
BEGIN
    -- SQL statements
END;
```

When to Use Which?

Use Case	Use Parameter
Provide input to a query	IN
Return computed value	OUT
Modify a value in place	INOUT

Simple Stored Procedure with an IN Parameter

Task: Display employee details for a given ID.

```
DELIMITER //

CREATE PROCEDURE GetEmployeeDetails(IN emp_id INT)
BEGIN
    SELECT employee_id, name, department, salary
    FROM employees
    WHERE employee_id = emp_id;
END;
//

DELIMITER ;
```

IN emp_id INT: Accepts an employee ID from the caller.

The SELECT query fetches and displays the details of that employee.

Calling the Stored Procedure:

```
CALL GetEmployeeDetails(102);
```

The procedure runs on the server, fetching details of employee ID 102.

Stored Procedure with IN and OUT Parameters

Task: Get salary of a specific employee and return it as output.

```
DELIMITER //

CREATE PROCEDURE GetSalary(IN emp_id INT, OUT emp_salary DECIMAL(10,2))
BEGIN
    SELECT salary INTO emp_salary
    FROM employees
    WHERE employee_id = emp_id;
END;
//

DELIMITER ;
```

Call It Like This:

```
CALL GetSalary(102, @sal);
SELECT @sal AS Salary;
```

Here:

@sal is a session variable to store the output.

The result is printed using a separate SELECT.

How to Declare and Set Variables in MySQL Stored Programs

- In MySQL, when you write procedural code (like stored procedures), variables allow you to temporarily store values and use them in calculations, conditions, loops, or output.
- Variables must be declared inside a BEGIN ... END block, typically in a stored procedure or function.

📐 Syntax to Declare a Variable

```sql
DECLARE variable_name data_type [DEFAULT value];
```

- `variable_name` : The name you assign to your variable.
- `data_type` : Data type (e.g., INT, DECIMAL, VARCHAR).
- `DEFAULT value` (optional): Initial value of the variable.

📌 How to Assign Values to Variables?

You can use either:

✅ `SET` Statement

```sql
SET variable_name = value;
```

✅ `SELECT INTO` Statement

Used when retrieving a value from a table into a variable.

```sql
SELECT column_name INTO variable_name FROM table_name WHERE condition;
```

How to Declare and Set Variables in MySQL Stored Programs

Task: To fetch the salary of an employee with ID 102

```
DELIMITER //

CREATE PROCEDURE FetchSalary()
BEGIN
    DECLARE total_salary DECIMAL(10,2);  -- Declare a variable

    SET total_salary = 50000;            -- Initialize with default or dummy value

    SELECT salary INTO total_salary      -- Fetch real value from DB
    FROM employees
    WHERE id = 102;

    SELECT total_salary AS 'Employee Salary';  -- Display value
END;
//

DELIMITER ;
```

Points to Remember

- DECLARE must be before any executable statement (like SET, SELECT).
- Variables are local to the BEGIN...END block in which they are declared.
- Always ensure that your SELECT INTO returns exactly one row — otherwise it may cause an error (NOT FOUND or Too many rows).

How to Display Data in Stored Programs

In procedural code (like stored procedures or functions), we often want to retrieve values from a table and store them into local variables so that we can use them later for logic, computation, or output.

For this purpose, MySQL uses SELECT ... INTO statement which assigns a column value from a query result into a declared variable.

Syntax:

```
SELECT column_name
INTO variable_name
FROM table_name
WHERE condition;
```

column_name: The name of the column whose value you want to retrieve.

variable_name: A declared local variable inside the procedure.

condition: Ensures the query returns only one row, or else an error is raised.

How to Display Data in Stored Programs

Task: To retrieve the name of an employee with ID 101:

```
DELIMITER //

CREATE PROCEDURE GetEmployeeName()
BEGIN
    DECLARE empName VARCHAR(50);

    SELECT name INTO empName
    FROM employees
    WHERE id = 101;

    SELECT empName AS 'Employee Name';
END;
//

DELIMITER ;
```

```
CALL GetEmployeeName();
```

Result:

```
+----------------+
| Employee Name  |
+----------------+
| John D'Souza   |
+----------------+
```

⚠ Important Notes

- The **SELECT INTO** statement must return **exactly one row**. If it returns:
  - **Zero rows** → you get a `NOT FOUND` error (can be handled using condition handlers).
  - **More than one row** → you get a `Too many rows` error.
- It is commonly used when working with cursors, loops, and business logic inside procedures.

How to Work with User Variables

What are User Variables

User-defined variables in MySQL are temporary variables that:

- Start with @
- Exist for the duration of the session
- Can store and pass data between SQL statements

Syntax

```
SET @var_name = value;

-- Or inside SELECT
SELECT column INTO @var_name FROM table WHERE condition;

-- Use it in expressions or queries
SELECT @var_name;
```

How to Work with User Variables

✅ Example 1: Using `SET`

```sql
SET @department = 'IT';
SELECT * FROM employees WHERE department = @department;
```

✅ Example 2: Using `SELECT ... INTO`

```sql
-- Assume 'employees' has at least one record with ID 101
SELECT name INTO @emp_name FROM employees WHERE employee_id = 101;
SELECT @emp_name AS "Employee Name";
```

How to Work with User Variables

✅ Example 3: Use in Procedure

```sql
DROP PROCEDURE IF EXISTS DemoUserVar;

DELIMITER //

CREATE PROCEDURE DemoUserVar()
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO @total_emp FROM employees;
    SELECT @total_emp AS "Total Employees";
END //

DELIMITER ;

CALL DemoUserVar();
```

✅ Example 4: Variable in Insert

```sql
SET @id = 103;
SET @name = 'Vishnu';
SET @dept = 'Finance';
SET @sal = 80000;

INSERT INTO employees (employee_id, name, department, salary)
VALUES (@id, @name, @dept, @sal);
```

🔍 Notes:

- User variables are **session-specific**.
- You don't need to declare them with a datatype.
- They're useful **outside** of stored routines, for quick testing and ad-hoc logic.

How to Set a Default value for a Parameter

- you can set a default value for a parameter in a stored procedure by assigning it a default during its declaration.
- This is useful when the user may or may not pass a value for that parameter when calling the procedure.

Syntax:

```
CREATE PROCEDURE procedure_name (
    IN param_name datatype DEFAULT default_value
)
BEGIN
    -- Procedure logic
END;
```

How to Set a Default value for a Parameter

```
DROP PROCEDURE IF EXISTS GetEmployeesByDept;

DELIMITER $$

CREATE PROCEDURE GetEmployeesByDept (
    IN dept_name VARCHAR(20) DEFAULT 'IT'
)
BEGIN
    SELECT * FROM employees WHERE department = dept_name;
END $$

DELIMITER ;
```

Output:

```
CALL GetEmployeesByDept('HR');
```

employee_id	name	department	salary
2	Rama	HR	55000.00

```
CALL GetEmployeesByDept(NULL);
```

employee_id	name	department	salary
1	Krishna	IT	60000.00
4	Lakshmi	IT	62000.00

How to Code IF Statements

The IF statement in MySQL is used to execute different blocks of code depending on conditions.

It works just like conditional statements in other programming languages.

Syntax :

```
IF condition THEN
    -- statements;
ELSEIF another_condition THEN
    -- statements;
ELSE
    -- statements;
END IF;
```

- `condition` : A logical expression that evaluates to TRUE or FALSE.
- `THEN` clause: Executes if the condition is true.
- `ELSEIF` : (optional) Additional checks.
- `ELSE` : (optional) Executes if no condition is true.
- `END IF` : Closes the IF block.

Stored Procedure to Insert a Row

Syntax:

```
DROP PROCEDURE IF EXISTS procedure_name;

DELIMITER //

CREATE PROCEDURE procedure_name (
    IN param1 DataType,
    IN param2 DataType,
    ...
)
BEGIN
    INSERT INTO table_name (column1, column2, ...)
    VALUES (param1, param2, ...);
END //

DELIMITER ;
```

Stored Procedure to Insert a Row

Example:

```
DROP PROCEDURE IF EXISTS InsertEmployee;

DELIMITER //

CREATE PROCEDURE InsertEmployee (
    IN emp_id INT,
    IN emp_name VARCHAR(50),
    IN emp_dept VARCHAR(50),
    IN emp_salary DECIMAL(10,2)
)
BEGIN
    INSERT INTO employees (employee_id, name, department, salary)
    VALUES (emp_id, emp_name, emp_dept, emp_salary);
END //

DELIMITER ;
```

Calling the Procedure

```
sql

CALL InsertEmployee(101, 'Ganesha', 'IT', 75000.00);
CALL InsertEmployee(102, 'Shiva', 'HR', 65000.00);
```

How to Code IF Statements

Task: Procedure that assigns a grade based on marks:

```
CREATE PROCEDURE AssignGrade(IN marks INT)
BEGIN
    DECLARE grade CHAR(1);

    IF marks >= 90 THEN
        SET grade = 'A';
    ELSEIF marks >= 75 THEN
        SET grade = 'B';
    ELSEIF marks >= 60 THEN
        SET grade = 'C';
    ELSE
        SET grade = 'D';
    END IF;

    SELECT grade AS 'Assigned Grade';
END;
//

DELIMITER ;
```

```
CALL AssignGrade(82);
```

Output

```
+----------------+
| Assigned Grade |
+----------------+
| B              |
+----------------+
```

⚠ Notes

- `IF` must be used **inside BEGIN...END** block.
- Each IF block must be closed with `END IF;`.
- Use `ELSEIF` (not `ELSE IF`) — it's one keyword in MySQL.

How to Validate Parameters and Raise Errors

In MySQL stored procedures, you can validate parameters using IF conditions and raise errors using the SIGNAL statement.

Syntax:

```
CREATE PROCEDURE procedure_name (IN param_name DATA_TYPE)
BEGIN
    -- Validation
    IF param_name IS NULL OR param_name = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid parameter: param_name cannot be NULL or empty';
    END IF;

    -- Main logic
    -- ...
END;
```

- `SIGNAL SQLSTATE '45000'` : Standard SQLSTATE for user-defined errors.
- `MESSAGE_TEXT` : Custom error message to display.

How to Validate Parameters and Raise Errors

Example: Validate Department Parameter

```
DELIMITER $$

CREATE PROCEDURE GetEmployeesByDept (
    IN dept_name VARCHAR(20)
)
BEGIN
    -- Parameter Validation
    IF dept_name IS NULL OR dept_name = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department name cannot be NULL or empty';
    END IF;

    -- Main Logic
    SELECT * FROM employees WHERE department = dept_name;
END $$

DELIMITER ;
```

✅ **Valid call:**

```
sql

CALL GetEmployeesByDept('IT');
```

❌ **Invalid call (NULL or empty string):**

```
sql

CALL GetEmployeesByDept('');
```

Output:

```
php

ERROR 1644 (45000): Department name cannot be NULL or empty
```

How to Code CASE Statements

The CASE statement is used when you need to evaluate multiple conditions and execute one block of code based on the first condition that is true.

It's similar to a switch-case structure in other programming languages.

1. Simple CASE (Matching a variable's value)

```sql
CASE expression
    WHEN value1 THEN statement1
    WHEN value2 THEN statement2
    ...
    ELSE default_statement
END CASE;
```

2. Searched CASE (Conditional expressions)

```sql
CASE
    WHEN condition1 THEN statement1
    WHEN condition2 THEN statement2
    ...
    ELSE default_statement
END CASE;
```

How to Code CASE Statements

🧪 Example 1: Simple CASE

```sql
DELIMITER //

CREATE PROCEDURE GradeCaseSimple(IN grade CHAR(1))
BEGIN
    DECLARE result VARCHAR(30);

    CASE grade
        WHEN 'A' THEN SET result = 'Excellent';
        WHEN 'B' THEN SET result = 'Good';
        WHEN 'C' THEN SET result = 'Average';
        ELSE SET result = 'Poor';
    END CASE;

    SELECT result AS 'Grade Description';
END;
//

DELIMITER ;
```

```sql
CALL GradeCaseSimple('B');
```

Output

```
+--------------------+
| Grade Description  |
+--------------------+
| Good               |
+--------------------+
```

✅ When to Use CASE Statements?

- When you have multiple `IF...ELSEIF` conditions.
- When choosing between outcomes based on a single value.
- To keep code clean and readable compared to nested `IF` statements.

How to Code CASE Statements

🧪 Example 2: Searched CASE

```sql
DELIMITER //

CREATE PROCEDURE TaxBracket(IN salary DECIMAL(10,2))
BEGIN
    DECLARE tax_rate DECIMAL(5,2);

    CASE
        WHEN salary >= 100000 THEN SET tax_rate = 30.0;
        WHEN salary >= 75000 THEN SET tax_rate = 20.0;
        WHEN salary >= 50000 THEN SET tax_rate = 10.0;
        ELSE SET tax_rate = 0.0;
    END CASE;

    SELECT tax_rate AS 'Applicable Tax Rate (%)';
END;
//

DELIMITER ;
```

```
CALL TaxBracket(85000);
```

Output

```
+-----------------------+
| Applicable Tax Rate (%)|
+-----------------------+
| 20.0                  |
+-----------------------+
```

⚠️ Notes

- Always **end the CASE block with** `END CASE;`.
- You can use either **simple** or **searched CASE** depending on your requirement.
- It's commonly used for grading, tiered pricing, category assignment, tax slabs, etc.

How to Work With Dynamic SQL

Dynamic SQL allows you to construct and execute SQL statements at runtime — helpful when:

- The table or column names are not known in advance.
- You want to conditionally build SQL queries inside stored procedures.

MySQL uses the PREPARE, EXECUTE, and DEALLOCATE PREPARE statements for dynamic SQL.

How to Work With Dynamic SQL

Syntax:

```
SET @sql = 'your SQL query as string';

PREPARE stmt_name FROM @sql;
EXECUTE stmt_name;
DEALLOCATE PREPARE stmt_name;
```

✅ Example: Select from a table dynamically

sql

```
-- Set a dynamic query to select all from 'employees'
SET @sql = 'SELECT * FROM employees WHERE department = "IT"';

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

How to Work With Dynamic SQL

✅ Example with Variable Table Name

```sql
SET @table = 'employees';
SET @sql = CONCAT('SELECT * FROM ', @table);

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

📝 Notes:

- Always **sanitize input** if you're using dynamic SQL to prevent SQL injection (important in applications).
- Use `CONCAT()` to build SQL strings dynamically.
- Dynamic SQL **cannot use INTO clause** for local variables inside `EXECUTE`; use user variables (`@`) instead if needed.

How to Work With Dynamic SQL

✅ Example Inside a Stored Procedure

```sql
DROP PROCEDURE IF EXISTS GetDynamicDept;

DELIMITER //

CREATE PROCEDURE GetDynamicDept(IN dept VARCHAR(20))
BEGIN
    SET @query = CONCAT('SELECT * FROM employees WHERE department = "', dept, '"');
    PREPARE stmt FROM @query;
    EXECUTE stmt;
    DEALLOCATE PREPARE stmt;
END //

DELIMITER ;

-- Call the procedure
CALL GetDynamicDept('HR');
```

How to Code Loops

MySQL supports three kinds of loops:

- WHILE
- REPEAT
- LOOP

Loops allow repetitive execution of a code block as long as a condition is true (or until it becomes true in the case of REPEAT).

How to Code Loops – While Loop

Syntax

```
WHILE condition DO
    -- statements;
END WHILE;
```

Example

```
DELIMITER //

CREATE PROCEDURE CountToTen()
BEGIN
    DECLARE counter INT DEFAULT 0;

    WHILE counter < 10 DO
        SET counter = counter + 1;
        SELECT counter AS "WHILE Loop Counter";
    END WHILE;
END;
//

DELIMITER ;
```

Output

The loop prints counter values from 1 to 10.

How to Code Loops – Repeat Loop

Syntax

```
REPEAT
    -- statements;
UNTIL condition
END REPEAT;
```

Difference:
Condition is **checked at the end**, so the body runs **at least once**.

Example

```
DELIMITER //

CREATE PROCEDURE CountDown()
BEGIN
    DECLARE counter INT DEFAULT 5;

    REPEAT
        SELECT counter AS "REPEAT Loop Counter";
        SET counter = counter - 1;
    UNTIL counter = 0
    END REPEAT;
END;
//

DELIMITER ;
```

Output

Prints counter values from 5 to 1.

How to Code Loops – Loop

Syntax

```
[loop_label:] LOOP
    -- statements
    IF condition THEN
        LEAVE loop_label;
    END IF;
END LOOP;
```

- loop_label is optional but needed if you plan to exit using LEAVE.
- Use LEAVE to break out of the loop when a condition is met.

Example

```
CREATE PROCEDURE DemoLoop()
BEGIN
    DECLARE counter INT DEFAULT 1;

    loop_label: LOOP
        IF counter > 5 THEN
            LEAVE loop_label;
        END IF;

        SELECT counter AS "LOOP Counter";
        SET counter = counter + 1;
    END LOOP;
END;
//
```

Output

```
+--------------+
| LOOP Counter |
+--------------+
| 1            |
| 2            |
| 3            |
| 4            |
| 5            |
+--------------+
```

Comparison of All Loop Types

Loop Type	Condition Position	Guaranteed 1 Execution	Exit Method
`WHILE`	Before the loop	❌	Condition becomes false
`REPEAT`	After the loop	✅	`UNTIL` condition is true
`LOOP`	No condition	✅ (manual exit)	Use `LEAVE` statement

✅ Use Case of `LOOP`

- Best used for complex logic where **manual control over exit** is needed.
- Helpful in cursor-based operations or nested conditions where early exit is desired.

Cursor in MySQL

A cursor in MySQL is a database object used to retrieve a set of rows (result set) row-by-row, which is especially useful when processing data iteratively in stored procedures.

Steps to Use a Cursor

1. **Declare the cursor** for a SELECT query.
2. **Open** the cursor.
3. **Fetch** rows from the cursor into variables.
4. **Repeat** fetches inside a loop.
5. **Close** the cursor.

Cursor in MySQL

Syntax for Using a Cursor:

```
DECLARE cursor_name CURSOR FOR
    SELECT_statement;

OPEN cursor_name;

FETCH cursor_name INTO variable1, variable2, ...;

CLOSE cursor_name;
```

Declare a handler to detect end of result set: (Needed to avoid runtime error when no more rows)

```
DECLARE done INT DEFAULT 0;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

Cursor in MySQL

Task: Displaying All Employee Names

```
DELIMITER //

CREATE PROCEDURE ListEmployees()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE emp_id INT;
    DECLARE emp_name VARCHAR(50);

    -- Declare cursor
    DECLARE emp_cursor CURSOR FOR
        SELECT id, name FROM employees;

    -- Declare handler to detect end of data
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    -- Open the cursor
    OPEN emp_cursor;

    read_loop: LOOP
        FETCH emp_cursor INTO emp_id, emp_name;

        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Display or process fetched data
        SELECT emp_id AS "Employee ID", emp_name AS "Employee Name";
    END LOOP;

    -- Close the cursor
    CLOSE emp_cursor;
END;
//

DELIMITER ;
```

Cursor in MySQL

Task: Displaying All Employee Names

Employee ID	Employee Name
101	Alice Kumar
102	Rahul Singh
103	Sneha Menon
104	John D'Souza
105	Priya Rao

⚠ Key Notes

Rule	Description
`CURSOR`	Must be declared **after variables**, before any SQL
`HANDLER`	Needed to avoid runtime error when no more rows
`OPEN`	Initializes the cursor
`FETCH`	Retrieves row into declared variables
`CLOSE`	Frees the resources

How to Use Multiple Condition Handlers

You can define multiple handlers for different error types within the same procedure.

Syntax:

```
DECLARE handler_type HANDLER
    FOR condition_value[, condition_value2, ...]
    statement;
```

- `handler_type`: `CONTINUE`, `EXIT`, or `UNDO` (only `CONTINUE` and `EXIT` are supported in MySQL).
- `condition_value`: Can be a **MySQL error code**, **SQLSTATE**, or **named condition**.
- `statement`: Code to execute when the condition is triggered.

How to Use Multiple Condition Handlers

Example: Using Multiple Condition Handlers

```
DROP PROCEDURE IF EXISTS ExampleWithHandlers;
DELIMITER //

CREATE PROCEDURE ExampleWithHandlers()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'General SQL Error occurred.';
    END;

    DECLARE CONTINUE HANDLER FOR SQLSTATE '23000'
    BEGIN
        SELECT 'Integrity constraint violation!';
    END;
    DECLARE CONTINUE HANDLER FOR NOT FOUND
    BEGIN
        SELECT 'No more rows to fetch!';
    END;

    -- Example code block that could raise errors
    DECLARE v_dummy INT;

    -- Intentional error: insert duplicate primary key (if applicable)
    INSERT INTO employees (employee_id, name, department, salary)
    VALUES (101, 'Duplicate Test', 'IT', 50000.00);  -- if 101 already exists

    SELECT 'Procedure completed.';
END //

DELIMITER ;
```

How to Drop a Stored Procedure

The DROP PROCEDURE statement is used to permanently delete a stored procedure from the database.

Syntax:

```
DROP PROCEDURE [IF EXISTS] procedure_name;
```

IF EXISTS is optional but recommended — it prevents errors if the procedure doesn't exist.

Example:

```
DROP PROCEDURE IF EXISTS GetEmployeesByDept;
```

This will drop the stored procedure named `GetEmployeesByDept` if it exists.

Tip: Before creating any procedure, always use:

DROP PROCEDURE IF EXISTS ProcedureName;

This avoids errors like: "Error Code: 1304. PROCEDURE already exists"

Stored Functions

A Stored Function (Scalar function) is a named block of SQL code stored in the database that:

- Takes input parameters
- Performs a computation or query
- Returns a single value
- It can be used within SQL statements, unlike stored procedures, which cannot
- It cant make changes to the database such as executing an INSERT, UPDATE or DELETE statement .

Stored Functions

🔄 Stored Function vs Stored Procedure

Feature	Stored Function	Stored Procedure
Returns a value?	✅ Yes (exactly one value)	🚫 No (uses OUT parameters instead)
Used in SELECT?	✅ Yes	🚫 No
Use in expressions?	✅ Yes	🚫 No
Purpose	Computation, return a value	Multiple operations, no return

📌 When to Use a Stored Function?

Use it when:

- You want to **re-use a calculation**
- You need a **return value** in queries
- You want **cleaner, modular SQL code**

Examples: tax calculations, discount logic, string processing, bonus % calculations, etc.

Creating a Stored Function

Syntax

```
DELIMITER //

CREATE FUNCTION function_name (
    parameter_name datatype
)
RETURNS return_datatype
DETERMINISTIC
BEGIN
    -- Declare local variables if needed
    -- Do calculations
    RETURN value;
END //

DELIMITER ;
```

Example

```
DELIMITER //

CREATE FUNCTION calculate_bonus (
    salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN salary * 0.10;
END //

DELIMITER ;
```

DETERMINISTIC: means the function always returns the same output for the same input (can also use NOT DETERMINISTIC).

Creating a Stored Function

Summary

- A Stored Function returns one value and can be called from SQL statements.
- It is ideal for reusable logic such as computing tax, bonus, or status based on inputs.
- It works inside SELECT, WHERE, ORDER BY, etc.
- You cannot use CALL to invoke a stored function (that's for stored procedures). Functions return values and must be used inside expressions.

Call the Function in a Query:

```sql
SELECT name, salary, calculate_bonus(salary) AS bonus
FROM employees;
```

Call in WHERE clause

```sql
SELECT *
FROM employees
WHERE calculate_bonus(salary) > 1000;
```

Creating a Stored Function

Example:

Suppose we created this function:

```sql
CREATE FUNCTION square(x INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN x * x;
END;
```

Now, to call it directly:

```sql
SELECT square(5) AS result;
```

Creating a Stored Function

🔍 What Does `DETERMINISTIC` Mean?

- ✅ `DETERMINISTIC`:
  The function **always returns the same result** for the same input.
- ❌ `NOT DETERMINISTIC`:
  The function **may return different results** even for the same input (e.g., it uses `NOW()`, `RAND()`, etc.).

✅ Example of `DETERMINISTIC` Function

```sql
CREATE FUNCTION square(x INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN x * x;
END;
```

Calling `square(4)` will **always return 16.**

So, it's **deterministic.**

❌ Example of `NOT DETERMINISTIC` Function

```sql
CREATE FUNCTION get_random_bonus()
RETURNS INT
NOT DETERMINISTIC
BEGIN
    RETURN FLOOR(RAND() * 100);
END;
```

Calling `get_random_bonus()` might return a **different value each time.**

So, it's **not deterministic.**

Dropping a Stored Function

✅ Syntax

```sql
DROP FUNCTION [IF EXISTS] function_name;
```

- `IF EXISTS` is **optional**. Use it to avoid an error if the function doesn't exist.

📌 Example 1: Drop function only if it exists

```sql
DROP FUNCTION IF EXISTS get_discounted_price;
```

This will:

- Drop the function `get_discounted_price` if it exists.
- **Do nothing (no error)** if the function does not exist.

📌 Example 2: Drop function without check

```sql
DROP FUNCTION get_discounted_price;
```

- If the function does **not exist**, MySQL will show an **error**:

```vbnet
ERROR 1305 (42000): FUNCTION get_discounted_price does not exist
```

Trigger in MySQL

A Trigger is a named database object that is automatically executed (or "triggered") in response to INSERT, UPDATE, or DELETE events on a table.

It’s often used to:

- Maintain audit trails
- Enforce business rules
- Validate or adjust data automatically

Trigger in MySQL

✅ Syntax: Create a Trigger

```sql
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- Trigger logic
END;
```

- `BEFORE` or `AFTER`: defines when the trigger fires.
- `INSERT`, `UPDATE`, `DELETE`: defines the triggering event.
- `FOR EACH ROW`: applies the trigger for every affected row.
- `NEW.column_name` and `OLD.column_name`: refer to column values before and after the change.

Trigger in MySQL – After Trigger Example

```sql
CREATE TABLE IF NOT EXISTS employee_log (
    emp_id INT,
    action VARCHAR(20),
    log_date DATETIME
);
```

Example: Auto-update log table when employee is added

```sql
CREATE TRIGGER log_new_employee
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log (emp_id, action, log_date)
    VALUES (NEW.employee_id, 'INSERTED', NOW());
END;
```

If the employee_log table does not exist,

Then

- The trigger will fail when it tries to run.
- The entire DML statement (e.g., INSERT INTO employees ...) that fired the trigger will be rolled back.
- You'll get an error like:

```
ERROR 1146 (42S02): Table 'your_database.employee_log' doesn't exist
```

Trigger in MySQL – After Trigger Example

`employees` Table (Before Insert)

employee_id	name	department	salary
101	Shiva	IT	50000.00

You Run This:

```sql
INSERT INTO employees (employee_id, name, department, salary)
VALUES (102, 'Lakshmi', 'HR', 60000.00);
```

Trigger Fires Automatically

This runs:

```sql
INSERT INTO employee_log (emp_id, action, log_date)
VALUES (102, 'INSERTED', NOW());
```

Sample `employee_log` Table (After Trigger Fires)

emp_id	action	log_date
102	INSERTED	2025-07-18 11:45:30

- `emp_id` : comes from `NEW.employee_id` = `102`
- `action` : hardcoded as `'INSERTED'`
- `log_date` : automatically takes current timestamp via `NOW()`

Trigger in MySQL – After Trigger Example

⚠️ Important Behavior

In MySQL:

- Triggers are part of the transaction logic.
- If any part of the trigger fails (due to missing table, data error, etc.), the original triggering statement is **aborted**.
- There is **no partial commit** — either everything succeeds, or nothing is saved.

✅ Final Notes

- The user sees **no output** from the trigger — it runs silently.
- But the **effect** is visible in the `employee_log` table as shown above.

📝 Summary

Situation	Outcome
`employee_log` exists	Trigger executes and logs data
`employee_log` missing or corrupted	Trigger fails, and DML is rolled back

Trigger in MySQL – Before Trigger Example

Example: Log the old salary before it's updated.

```
CREATE TRIGGER before_employee_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log (emp_id, action, log_date)
    VALUES (OLD.employee_id, CONCAT('OLD SALARY: ', OLD.salary), NOW());
END;
```

Update Operation

```
UPDATE employees
SET salary = 75000
WHERE employee_id = 101;
```

Sample Log Entry:

emp_id	action	log_date
101	OLD SALARY: 50000.00	2025-07-18 12:30:00

View All Triggers in Current Database

```
SHOW TRIGGERS;
```

This shows a table with:

- Trigger name
- Event (INSERT, UPDATE, DELETE)
- Timing (BEFORE/AFTER)
- Table it's attached to
- Statement and other metadata

Trigger Name	Event	Table	Statement	Timing
after_employee_insert	INSERT	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (NEW.employee_id, 'INSERTED', NOW())	AFTER
before_employee_update	UPDATE	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (OLD.employee_id, CONCAT('OLD SALARY: ', OLD.salary), NOW())	BEFORE

View Triggers for a Specific Table

```
SHOW TRIGGERS LIKE 'employees';
```

Trigger Name	Event	Table	Statement	Timing
after_employee_insert	INSERT	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (NEW.employee_id, 'INSERTED', NOW())	AFTER
before_employee_update	UPDATE	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (OLD.employee_id, CONCAT('OLD SALARY: ', OLD.salary), NOW())	BEFORE
before_employee_delete	DELETE	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (OLD.employee_id, 'DELETED', NOW())	BEFORE

Drop a Trigger

```
DROP TRIGGER [IF EXISTS] trigger_name;
```

- `IF EXISTS` is optional but prevents errors if the trigger doesn't exist.

🧪 Example: Drop an existing trigger

```
sql

DROP TRIGGER IF EXISTS before_employee_update;
```

This safely removes the trigger without giving an error if it doesn't exist.

🧠 Notes:

- Triggers are tied to **tables**, not procedures or functions.
- You **cannot modify** a trigger — you must drop and recreate it.
- Triggers are stored **per database**, so ensure you're in the right schema.

Transactions

- A Transaction is a group of SQL statements that are executed as a single unit.
- By default, MYSQL runs in a auto commit mode, which automatically commits changes to the database immediately after each INSERT, UPDATE or DELETE statement is executed.
- To start a transaction, code the START TRANSACTION statement. This turns off auto commit mode until the statements in the transaction are committed or rolled back. To commit the changes, code a COMMIT statement. To rollback the changes, use a ROLLBACK statement.
- MySQL automatically commits changes after a DDL statement such as a CREATE TABLE statement. As a result you shouldn't code a DDL statement within a transaction unless you want to commit the changes and end the transaction.

Transactions

Syntax:

```
START TRANSACTION;
-- SQL statements
COMMIT;     -- save changes
-- OR
ROLLBACK;   -- undo changes
```

Feature	Command	Purpose
Start Txn	`START TRANSACTION;`	Begin a transaction
Lock Row	`SELECT ... FOR UPDATE;`	Lock specific rows
Save Changes	`COMMIT;`	Make all changes permanent
Undo Changes	`ROLLBACK;`	Undo all changes
Savepoint	`SAVEPOINT name;`	Create rollback checkpoint
Partial Undo	`ROLLBACK TO SAVEPOINT;`	Rollback to a savepoint

How to Work with Savepoints

SAVEPOINT allows you to create checkpoints inside a transaction.

You can rollback to a SAVEPOINT instead of undoing the entire transaction.

```
START TRANSACTION;

SAVEPOINT savepoint_name;

-- SQL statements

ROLLBACK TO SAVEPOINT savepoint_name;

COMMIT;
```

```
START TRANSACTION;

UPDATE employees SET salary = salary + 500 WHERE emp_id = 1;
SAVEPOINT after_first_update;

UPDATE employees SET salary = salary + 1000 WHERE emp_id = 2;
SAVEPOINT after_second_update;

-- Cancel only second update
ROLLBACK TO SAVEPOINT after_second_update;

-- Continue with other operations
UPDATE employees SET department = 'Finance' WHERE emp_id = 1;

COMMIT;
```

How to Work with Concurrency and Locking

- Concurrency means multiple users accessing the database at the same time.
- To manage it safely, locking is used to avoid conflicts.
- Concurrency enables performance and scalability.
- Locking ensures data consistency during concurrent access.
- Without locking, dirty reads, lost updates, and data corruption can occur.

Types of Locks in MySQL:

Lock Type	Description
Shared (Read)	Allows multiple reads
Exclusive (Write)	Only one write allowed; blocks other reads/writes

How to Work with Concurrency and Locking

Syntax:

```
LOCK TABLES table_name [AS alias]
    READ | WRITE;

-- Do your read/write operations here

UNLOCK TABLES;
```

🔒 Step-by-step Locking Example

```sql
-- Step 1: Lock table for writing
LOCK TABLES employees WRITE;

-- Step 2: Update salary (simulate a long-running operation)
UPDATE employees SET salary = salary + 1000 WHERE emp_id = 1;

-- Step 3: Unlock when done
UNLOCK TABLES;
```

⏱ During this time, **other sessions cannot read/write** the `employees` table.

How to Work with Concurrency and Locking

🔒 Shared (READ) Lock Example

```sql
LOCK TABLES employees READ;

-- Safe: SELECTs are allowed from others
SELECT * FROM employees;

UNLOCK TABLES;
```

⚠️ Notes:

- Lock is **session-specific**: only affects current connection.
- While a **WRITE lock** is held, other sessions **cannot even read**.
- Used in manual locking or complex transaction control.

The Four Concurrency Problems Locks Can Prevent

Problem	Description
1. Dirty Read	A transaction reads uncommitted data from another transaction.
2. Non-Repeatable Read	Same query returns different data if another transaction modifies it.
3. Phantom Read	New rows appear in a second read due to another transaction's INSERT.
4. Lost Update	Two transactions read, modify, and write back at the same time, losing data.

The Four Concurrency Problems Locks Can Prevent

✅ Four Main Concurrency Problems

Problem	Description	Example
Dirty Read	Reading uncommitted changes from another transaction.	T1 updates a value, T2 reads it before T1 commits/rolls back.
Non-repeatable Read	Reading the same row twice and getting different results because another transaction committed changes between reads.	T1 reads a row, T2 updates it and commits, then T1 reads again.
Phantom Read	Re-running a query returns different rows due to inserts/deletes by another transaction.	T1 reads rows with `WHERE dept='IT'`, T2 inserts a new 'IT' employee.
Lost Update	Two transactions read the same value and both update it, but one update overwrites the other without knowing.	T1 and T2 read salary = 1000, both add 200, but last commit overwrites first.

How to Set the Transaction Isolation Level

The transaction isolation level controls the degree to which transactions are isolated from one another.

At the more restrictive isolation levels, concurrency problems are reduced or eliminated.

However, at the least restrictive levels, performance is enhanced.

To change the transaction isolation level, you use the SET TRANSACTION ISOLATION LEVEL statement.

Syntax:

```
-- Set globally (server-wide)
SET GLOBAL TRANSACTION ISOLATION LEVEL [LEVEL];

-- Set for current session
SET SESSION TRANSACTION ISOLATION LEVEL [LEVEL];
```

How to Set the Transaction Isolation Level

✅ Transaction Isolation Levels vs Concurrency Problems

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read	Lost Update
READ UNCOMMITTED	❌ Allowed	❌ Allowed	❌ Allowed	❌ Allowed
READ COMMITTED	✅ Prevented	❌ Allowed	❌ Allowed	❌ Allowed
REPEATABLE READ	✅ Prevented	✅ Prevented	❌ Allowed	❌ Allowed
SERIALIZABLE	✅ Prevented	✅ Prevented	✅ Prevented	✅ Prevented

✅ = Prevented

❌ = Possible

> **Note:** In MySQL (InnoDB), **Lost Updates** can still happen in `READ COMMITTED` and `REPEATABLE READ` unless you use **explicit locking** (like `SELECT ... FOR UPDATE`) or **optimistic concurrency control**.

How to Set the Transaction Isolation Level

- If you include the **GLOBAL** keyword, the isolation level is set globally for all new transactions in all sessions. If you include the **SESSION** keyword, the isolation level is set for all new transactions in the current session. If you omit both **GLOBAL** and **SESSION**, the isolation level is set for the next new transaction in the current session.
- The default transaction isolation level is **REPEATABLE READ**. This level places locks on all data that's used in a transaction, preventing other users from updating that data. However, this isolation level still allows inserts, so phantom reads can occur.
- The **READ UNCOMMITTED** isolation level doesn't set any locks and ignores locks that are already held. This level results in the highest possible performance of your query, but at the risk of every kind of concurrency problem. For this reason, you should only use this level for data that is rarely updated.
- The **READ COMMITTED** isolation level locks data that has been changed but not committed. This prevents dirty reads but allows all other types of concurrency problems.
- The **SERIALIZABLE** isolation level places a lock on all data that's used in a transaction. Since each transaction must wait for the previous transaction to commit, the transactions are handled in sequence. This is the most restrictive isolation level.

How to Set the Transaction Isolation Level

Example:

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

SELECT * FROM employees WHERE dept = 'IT';  -- snapshot taken here

-- another session updates an 'IT' employee, but this transaction will not see the change

UPDATE employees SET salary = salary + 5000 WHERE dept = 'IT';

COMMIT;
```

Even if someone else updates an employee in 'IT' after your `SELECT`, your update will work based on the snapshot taken when the transaction started.

How to Set the Transaction Isolation Level

Example:

⚠ Important:

To **prevent lost updates**, use:

```sql
SELECT * FROM employees WHERE id = 101 FOR UPDATE;
```

This locks the row so that others can't write to it until your transaction ends.

How to Lock Selected Rows

Use SELECT ... FOR UPDATE or LOCK IN SHARE MODE (InnoDB only):

For Update (Write Lock):

```sql
START TRANSACTION;
SELECT * FROM employees WHERE emp_id = 1 FOR UPDATE;
-- This row is locked for writing
UPDATE employees SET salary = salary + 500 WHERE emp_id = 1;
COMMIT;
```

Shared Lock (Read Lock):

```sql
START TRANSACTION;
SELECT * FROM employees WHERE emp_id = 1 LOCK IN SHARE MODE;
-- Allows other reads, prevents writes
COMMIT;
```

How to Prevent Deadlocks

Deadlock happens when two transactions wait indefinitely for each other to release locks.

Prevention Techniques:

- **Access tables in same order** in all transactions.
- **Keep transactions short and fast.**
- **Use appropriate isolation levels** (not too strict).
- **Use indexing** to minimize locked rows.

Example of Safe Access Order:

```sql
-- Transaction A
START TRANSACTION;
SELECT * FROM employees WHERE emp_id = 1 FOR UPDATE;
UPDATE employees SET salary = salary + 1000 WHERE emp_id = 1;
COMMIT;

-- Transaction B (access same row after A commits)
START TRANSACTION;
SELECT * FROM employees WHERE emp_id = 1 FOR UPDATE;
UPDATE employees SET department = 'HR' WHERE emp_id = 1;
COMMIT;
```